ECE 562: Deep Reinforcement Learning for Robotics

Fall 2026

Assignment 4: Soft Actor-Critic

Instructor:	Jun Jin (jun.jin@ualberta.ca)
TA:	Arya Ebrahimi (ebrahim5@ualberta.ca)
Created on:	October 26, 2026
Updated on:	N/A
Deadline:	November 16, 2026, at 11:59 p.m. (Edmonton time)

In Assignment 3 you saw that PPO can only use data that is a few iterations old. Its importance ratio $r_\theta(s, a)$ corrects for the change in the action distribution but not for the change in the state distribution, and as the data gets older more and more samples end up outside the clip range. Soft Actor-Critic (SAC) works very differently. It keeps a replay buffer that, by the end of training, holds data from hundreds of thousands of steps ago, collected by many different past policies. It still learns well from this data, and it never computes an importance ratio.

The goal of this assignment is to find out why. You will first implement SAC yourself. You will then measure how off-policy its data really is, and replace the key parts of its update with the PPO-style alternatives from Assignment 3 to see which part lets it learn from old data.

There is no theory section in this assignment. It has two parts.

- **Implementing SAC (30 points).** As in Assignment 2, you complete the marked TODO sections of a SAC codebase for Hopper-v5, the same environment as in Assignment 3. You will write the tanh-squashed Gaussian policy, the twin-critic update, the actor update, and the temperature and target-network updates.
- **Experiments and Report (70 points).** Using your SAC, you add some diagnostics and then swap parts of the SAC update for PPO-style versions.

To get full credit, describe your code changes clearly, include every plot that is asked for, and back up each empirical claim with your results. When a question refers to Assignment 3, connect your answer to the specific question there. If a result is not what you expected, report it anyway and explain what might have caused it.

Notation. We use the following notation throughout.

- The SAC actor is a tanh-squashed Gaussian. It first samples a *pre-tanh action*

$$u \sim \mathcal{N}\big(\mu_\theta(s), \mathrm{diag}(\sigma_\theta(s)^2)\big)$$

and then sends $a = \tanh(u) \in (-1, 1)^3$ to the environment. We write $\pi_\theta(a \mid s)$ for the density of the squashed action and $\mathcal{N}_\theta(u \mid s)$ for the density of the pre-tanh action. The two are related by

$$\log \pi_\theta(a \mid s) = \log \mathcal{N}_\theta(u \mid s) - \sum_{i=1}^{3} \log\big(1 - \tanh^2 u_i\big).$$

- $\mathcal{D}$ is the replay buffer. Every stored transition (s, a, r, s', d) was collected by some past policy, which we call its *behavior policy* π_b. Different transitions have different behavior policies. During the first 5,000 steps (the *warm-up*) actions are drawn uniformly from $[-1, 1]^3$. After that, the behavior policy of a transition is the actor π_θ as it was when the transition was collected.
- If a transition was collected at environment step t_c and is used in an update at step t, its *data age* is $t - t_c$. One PPO iteration in Assignment 3 was 2048 environment steps, so a data age of $t - t_c$ is the same as a lag of $k = (t - t_c)/2048$ PPO iterations.
- The importance ratio of a stored transition is $r_\theta(s, a) = \pi_\theta(a \mid s)/\pi_b(a \mid s)$. The tanh term above is the same for both policies, so it cancels and

$$\log r_\theta(s, a) = \log \mathcal{N}_\theta(u \mid s) - \log \mathcal{N}_b(u \mid s).$$

- $\bar{Q}(s, a) = \min\big(Q_{\phi_1}(s, a), Q_{\phi_2}(s, a)\big)$ is the smaller of the two critics, and $\bar{Q}_{\text{targ}}$ is the same quantity computed with the target critics. α is the entropy temperature.

Submission Instructions. Submit the following through Canvas.

- `report.pdf` with the output of `python test_sac.py`, a description of your code changes, your plots, and your written analysis. You can write the report in LaTeX or by hand, as long as the scanned PDF is clear.
- `code/` with all TODO sections completed and all the changes from Section 2. Include a short `README.md` that lists the exact commands that produce every figure in your report. Your code must run without errors.

1 Implementing SAC (30 points)

Data collection, the replay buffer, evaluation and logging are already written for you. You need to complete five TODO sections, TODO 1 in `networks.py` and TODOs 2, 3, 4a and 4b in `sac.py`. Once they are done, `python train.py` trains standard SAC. Here is what each file contains.

- `networks.py` has the tanh-Gaussian actor and the Q-network, both with 2 hidden layers of 256 ReLU units. The actor's $\log \sigma$ depends on the state and is clipped to $[-5, 2]$. `Actor.sample` (TODO 1) returns a reparameterized sample along with u, $\log \pi_\theta(a \mid s)$, $\log \mathcal{N}_\theta(u \mid s)$, $\mu_\theta(s)$ and $\log \sigma_\theta(s)$. `Actor.log_prob_u` gives $\log \mathcal{N}_\theta(u \mid s)$ for a pre-tanh action you pass in, and `tanh_log_det_jacobian` computes $\sum_i \log(1 - \tanh^2 u_i)$ in a numerically stable way.
- `replay_buffer.py` is a FIFO replay buffer. Besides (s, a, r, s', d), it records for each transition the pre-tanh action u, the behavior policy's $\mu_b(s)$ and $\log \sigma_b(s)$, the behavior log-density $\log \mathcal{N}_b(u \mid s)$ (`beh_logp_u`), the step at which it was collected (`step`), and whether it came from the warm-up (`is_random`). For warm-up transitions, `beh_logp_u` is the exact pre-tanh density of the uniform policy, and `beh_mu` and `beh_log_std` are just placeholders.
- `sac.py` contains `SACAgent.update()`, which runs the optimizer steps for the critics, the actor and the temperature, in that order, and then updates the target networks. It calls the four methods you write, `critic_loss` (TODO 2), `actor_loss` (TODO 3), `alpha_loss` (TODO 4a) and `soft_update` (TODO 4b).

- `test_sac.py` has unit tests for each TODO. Passing them does not prove your code is correct, but if one fails, something is wrong.
- `train.py` handles data collection (uniform warm-up, then the stochastic actor), one update per environment step, deterministic evaluation, and logging. Every 5,000 environment steps it writes a row to `runs/<run-name>/seed<seed>/progress.csv` with the evaluation return and the average of every value returned by `SACAgent.update()` since the last row. This means any new key you add to that dictionary shows up in the CSV automatically.
- `plot.py` plots the mean across seeds, with a shaded band of one standard deviation, for any logged column and any number of run folders. It works with your Assignment 3 runs too.

Both `critic_loss` and `actor_loss` must get their actions from *exactly one* call to `self.actor.sample`, because the tests depend on it. In this section, do not change `update()`, the buffer or the training loop.

Q1 (8 points) The Tanh-Squashed Gaussian Policy (`networks.py`)

Complete TODO 1 in `Actor.sample`. The method `dist_params` already gives you $\mu_\theta(s)$ and the clipped $\log \sigma_\theta(s)$, each with shape `(batch_size, 3)`.

(a) **(5 points)** Build the pre-tanh Gaussian with `torch.distributions.Normal`, using $\sigma_\theta(s) = \exp(\log \sigma_\theta(s))$, and draw a *reparameterized* sample

$$u = \mu_\theta(s) + \sigma_\theta(s) \odot \xi, \qquad \xi \sim \mathcal{N}(0, I).$$

Use `rsample` so that gradients can flow from u back to θ. The action is $a = \tanh(u)$.

(b) **(3 points)** Compute the pre-tanh and squashed log-densities. Both should have shape `(batch_size,)`.

$$\log \mathcal{N}_\theta(u \mid s) = \sum_{i=1}^{3} \log \mathcal{N}\big(u_i \mid \mu_{\theta,i}(s), \sigma_{\theta,i}(s)\big)$$

$$\log \pi_\theta(a \mid s) = \log \mathcal{N}_\theta(u \mid s) - \sum_{i=1}^{3} \log\left(1 - \tanh^2 u_i\right)$$

Use the provided `tanh_log_det_jacobian(u)` for the last sum.

Q2 (10 points) The Soft Bellman Critic Update (`sac.py`)

Complete TODO 2 in `SACAgent.critic_loss`. The batch gives you the stored states s, actions a, rewards r, next states s' and termination flags d (`terminated`). Episodes that end because of the time limit are stored with $d = 0$, so they are still bootstrapped.

(a) **(6 points)** Inside `torch.no_grad()`, sample $a' \sim \pi_\theta(\cdot \mid s')$ with one call to `self.actor.sample` and compute the soft Bellman target with the *target* critics,

$$y = r + \gamma(1 - d)\Big[\min_{j=1,2} Q_{\bar{\phi}_j}(s', a') - \alpha \log \pi_\theta(a' \mid s')\Big].$$

(b) **(4 points)** Evaluate both online critics on the *stored* actions. Return the scalar loss below together with $Q_{\phi_1}(s, a)$, which is used for logging.

$$L_Q(\phi_1, \phi_2) = \tfrac{1}{2}\,\hat{\mathbb{E}}\big[(Q_{\phi_1}(s,a) - y)^2\big] + \tfrac{1}{2}\,\hat{\mathbb{E}}\big[(Q_{\phi_2}(s,a) - y)^2\big]$$

Q3 (12 points) Actor, Temperature, and Target-Network Updates (`sac.py`)

(a) **(5 points)** Complete TODO 3 in `SACAgent.actor_loss`. Draw fresh actions $\tilde{a} \sim \pi_\theta(\cdot \mid s)$ with one call to `self.actor.sample`. The sample is reparameterized, so the gradient flows through $\tilde{a}$. Return the scalar loss below together with $\log \pi_\theta(\tilde{a} \mid s)$.

$$L_\pi(\theta) = \hat{\mathbb{E}}_{s\sim\mathcal{D}}\Big[\alpha \log \pi_\theta(\tilde{a} \mid s) - \min_{j=1,2} Q_{\phi_j}(s, \tilde{a})\Big]$$

Use the *online* critics here, and treat α as a constant (`self.alpha` is already detached).

(b) **(4 points)** Complete TODO 4a in `SACAgent.alpha_loss`. The target entropy is $\bar{\mathcal{H}} = -3$ (`self.target_entropy`), and the loss is

$$L(\alpha) = -\hat{\mathbb{E}}\Big[\log \alpha \,\big(\log \pi_\theta(\tilde{a} \mid s) + \bar{\mathcal{H}}\big)\Big].$$

Treat $\log \pi_\theta(\tilde{a} \mid s)$ as a constant so that only `self.log_alpha` gets a gradient.

(c) **(3 points)** Complete TODO 4b in `SACAgent.soft_update`. For both critics, apply the Polyak update $\bar{\phi}_j \leftarrow (1-\tau)\,\bar{\phi}_j + \tau\,\phi_j$ with $\tau = 0.005$ (`self.tau`).

When you are done, run `python test_sac.py` and put its output in your report. In Q4(b) you will also check that your implementation actually learns.

2 Experiments and Report (70 points)

Now you will change your completed code. You can organize the changes however you like, and we suggest adding the command-line flags named below. Please implement each change exactly as described so that results can be compared across students. Do not change the default hyperparameters.

Total environment steps	2×10^5	Discount γ	0.99
Warm-up steps (uniform)	5,000	Target smoothing τ	0.005
Buffer capacity	10^6	Learning rates (actor, critics, α)	3×10^{-4}
Batch size	256	Initial temperature α	1.0
Updates per env. step	1	Target entropy	-3
Hidden layers	2×256, ReLU	Evaluation interval	5,000 steps
		Evaluation episodes	5

The buffer capacity is larger than the total number of steps, so nothing is ever thrown away. Unlike the Assignment 3 code, there is no observation or reward normalization. Evaluation uses the deterministic action $\tanh(\mu_\theta(s))$.

Run every configuration for 2×10^5 environment steps with seeds $\{1, 2, 3\}$. Every learning curve should show the mean across seeds with a shaded band of one standard deviation, with environment steps on the x-axis. One run takes about 30 to 60 minutes on a single CPU core (the code uses one thread by default), and runs are independent, so you can run them in parallel. There are 4 configurations in total, which means 12 runs, so please start early.

Some plots compare against your Assignment 3 results. Reuse your Assignment 3 Q5 runs of PPO with $k \in \{0, 2, 4\}$, which all used the `clip` loss. If you no longer have them, rerun them with your Assignment 3 code. They take 5 to 15 minutes each.

Q4 (28 points) Measuring How Off-Policy SAC's Data Is

(a) **(10 points) Modification.** Add the diagnostics listed below. Compute them on the batch sampled for each update, *before* any parameter changes and without gradients, and return them from `SACAgent.update()` so they get logged. Computing them must not affect training in any way.

- `data_age_mean` is the mean data age of the batch in environment steps, over all samples.
- `kl_beh` is the mean, over non-warm-up samples, of $D_{\mathrm{KL}}\big(\mathcal{N}_b(\cdot \mid s) \,\big\|\, \mathcal{N}_\theta(\cdot \mid s)\big)$. Here $\mathcal{N}_b$ is the stored behavior Gaussian with parameters $\mu_b(s)$ and $\sigma_b(s)$, and $\mathcal{N}_\theta$ is the current actor's Gaussian at the same state. For diagonal Gaussians,

$$D_{\mathrm{KL}}\big(\mathcal{N}_b \,\|\, \mathcal{N}_\theta\big) = \sum_{i=1}^{3} \left[\log \frac{\sigma_{\theta,i}}{\sigma_{b,i}} + \frac{\sigma_{b,i}^2 + (\mu_{b,i} - \mu_{\theta,i})^2}{2\sigma_{\theta,i}^2} - \frac{1}{2} \right].$$

- `log_ratio_p05`, `log_ratio_p50` and `log_ratio_p95` are the 5th, 50th and 95th percentiles of $\log r_\theta(s, a)$ over non-warm-up samples, computed from the stored pre-tanh action u and `beh_logp_u`.
- `clipfrac_beh` is the fraction of non-warm-up samples with $r_\theta(s, a) \notin [1 - \epsilon, 1 + \epsilon]$ for $\epsilon = 0.2$. It is the clip fraction PPO would report if it trained on this batch. Compute it from $\log r_\theta$, since the ratios of old data can overflow.

In your report, describe how you implemented these. Briefly explain why it is fine to measure the KL between the pre-tanh Gaussians instead of the squashed policies, and why warm-up samples are left out of `kl_beh`.

(b) **(6 points) Results.** Train your completed SAC. We call this run `base`, and you will reuse it in Q5. It also tells you whether your implementation works. A single seed can sit at a return of 300 to 400 for a while (the hopper keeps its balance but does not move forward), but if all three seeds stay there for the whole run, go back and check your TODOs. Make the following plots.

- The evaluation return of SAC together with your Assignment 3 PPO run with $k = 0$.
- `data_age_mean` converted to PPO iterations (divide by 2048), with horizontal reference lines at $k = 2$ and $k = 4$.
- `kl_beh` together with `kl_start` from your Assignment 3 runs with $k \in \{0, 2, 4\}$, on a logarithmic y-axis.

- `clipfrac_beh` together with `clipfrac_start` from your Assignment 3 runs with $k \in \{0, 2, 4\}$.

(c) **(12 points) Analysis.** Compare how sample efficient SAC and PPO are at 10^5 and 2×10^5 environment steps. How old, in PPO iterations, is the average transition SAC learns from? How does that compare with the lag k at which PPO started to struggle in Assignment 3? How different is SAC's behavior policy from its current policy, judging by `kl_beh` and `clipfrac_beh`, and how does that compare with PPO? Using your answer to Assignment 3 Q5(c), roughly what fraction of SAC's samples would still give a non-zero policy gradient if SAC used PPO's clipped loss on its replay data?

SAC never computes r_θ. Go through `sac.py` and list every quantity in the critic and actor updates that comes from the replay buffer, and every action that is freshly sampled from the current policy. From this list, make a guess about how SAC gets away without an importance ratio. You will test your guess in Q5.

Q5 (42 points) Replacing Fresh Actions with Stored Ones

In Assignment 3 Q3(b) you found that PPO makes two approximations when it reuses data from $\pi_{\theta_{\text{old}}}$. It ignores the state-distribution ratio, $w_\theta(s) \approx 1$, and it uses the advantage of the old policy in place of the current one, $A^{\pi_\theta} \approx A^{\pi_{\theta_{\text{old}}}}$. SAC samples actions from the current policy in two places. In this question you replace each of them with actions from the replay buffer and see what happens.

(a) **(8 points) Modification of the critic target.** Add a choice of next action in the soft Bellman target (suggested flag `--target-action`). The target is

$$y = r + \gamma(1 - d)\Big[\bar{Q}_{\text{targ}}(s', a') - \alpha \log \pi(a' \mid s')\Big].$$

In the `current` variant, which is your TODO 2, $a' \sim \pi_\theta(\cdot \mid s')$ and $\pi = \pi_\theta$. In the `replay` variant, $a' = a'_b$ is the action the behavior policy actually took at s', which is the action of the next transition in the same episode, and $\pi = \pi_b$. Here $\log \pi_b(a'_b \mid s')$ is the squashed log-density of that action under the behavior policy, which you can get with `tanh_log_det_jacobian`. To implement this, store the next action and its behavior log-density with each transition once the agent picks it. If there is no next action (the episode was cut off by the time limit, or the next step has not been collected yet), fall back to $a' \sim \pi_\theta(\cdot \mid s')$. For terminal transitions the next action does not matter, because $d = 1$.

In your report, describe how you implemented this. In the replay variant, whose value is the critic learning? Which of the two approximations from Assignment 3 Q3(b) does this variant bring back?

(b) **(8 points) Modification of the actor loss.** Add a choice of actor loss (suggested flag `--actor-loss`). Let (s, a) be a stored state-action pair from the batch with pre-tanh action u, and let $\tilde{a} \sim \pi_\theta(\cdot \mid s)$ be a fresh reparameterized sample. The critic-based advantage of the stored action is

$$\hat{A}(s, a) = \bar{Q}(s, a) - \bar{Q}(s, \tilde{a}).$$

Compute it without gradients and normalize it over the batch exactly as in Assignment 3, giving $(\hat{A} - \mu_{\hat{A}})/(\sigma_{\hat{A}} + 10^{-8})$. Then implement the three losses below, all to be minimized.

The first one is the loss you already wrote in TODO 3.

$$
\begin{aligned}
L_{\text{reparam}} &= \hat{\mathbb{E}}\big[\alpha \log \pi_\theta(\tilde{a} \mid s) - \bar{Q}(s, \tilde{a})\big] \\
L_{\text{naive}} &= -\hat{\mathbb{E}}\big[\log \pi_\theta(a \mid s)\, \hat{A}\big] + \alpha\, \hat{\mathbb{E}}\big[\log \pi_\theta(\tilde{a} \mid s)\big] \\
L_{\text{clip}} &= -\hat{\mathbb{E}}\Big[\min\big(r_\theta \hat{A},\ \text{clip}(r_\theta, 1-\epsilon, 1+\epsilon)\hat{A}\big)\Big] + \alpha\, \hat{\mathbb{E}}\big[\log \pi_\theta(\tilde{a} \mid s)\big]
\end{aligned}
$$

Use $\epsilon = 0.2$ and compute r_θ from `beh_logp_u`. The `naive` and `clip` losses are the ones from Assignment 3 Q6, with the stored behavior policy in place of $\pi_{\theta_{\text{old}}}$ and a critic-based advantage in place of GAE. The entropy term is the same as in SAC. The term $\sum_i \log(1 - \tanh^2 u_i)$ does not depend on θ, so you can use `Actor.log_prob_u` for $\log \pi_\theta(a \mid s)$ in the gradient. Leave the critic and temperature updates as they are. For `clip`, also log the fraction of samples in the actor batch with $r_\theta \notin [1-\epsilon, 1+\epsilon]$ as `clipfrac_actor`.

In your report, describe how you implemented this. For each of the three losses, say which part of the actor's input comes from the behavior policy and which part comes from the current policy.

(c) **(10 points) Results.** Keep every other setting at its default and train the three new configurations `--target-action replay`, `--actor-loss naive` and `--actor-loss clip`. Compare them with `base` from Q4. Make the following plots, with one curve per configuration.

- The evaluation return.
- The mean Q-value on the sampled batch (`q_mean`).
- The policy entropy (`entropy`).
- The temperature (`alpha`) on a logarithmic y-axis.

In a separate plot, show `clipfrac_actor` for the `clip` configuration together with `clipfrac_beh` for `base`. If a configuration becomes numerically unstable because of the objective itself, report when it happens and use the results you have up to that point.

(d) **(16 points) Analysis.** Rank the four configurations and explain why they differ.

- The actor in the replay-target configuration still uses fresh actions. Why does it matter whose value the critic learns? Use `q_mean` in your answer.
- In `naive`, which distribution do the actions in the policy gradient come from, and which distribution should they come from? Relate this to the missing correction in Assignment 3 Q6. If this configuration diverges, use `entropy` and `alpha` to explain what the policy is doing. Explain why minimizing $-\log \pi_\theta(a \mid s)\, \hat{A}$ over *fixed* stored actions can run off to infinity, and why the same loss did not diverge on fresh data in Assignment 3 with $k = 0$.
- The ratio in `clip` does correct the action distribution. So why does it still do worse than `reparam`? Use `clipfrac_actor` and your answer to Assignment 3 Q5(c).

Finally, go back to your guess from Q4(c). In a short paragraph, explain why SAC can sample fresh actions from the current policy instead of reweighting the stored ones, and why PPO cannot do the same, given that its critic is a state-value function $V(s)$ and its advantages come from GAE on the behavior trajectories. Based on Assignments 3 and 4, what does each algorithm gain and what does it pay for its choice?